

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Mateus dos Santos Pereira

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	2
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	10
4.5	Atividade 5 — Logs e auditoria	13
4.6	Atividade 6 — Upload e perfil de usuário	15
5	Considerações finais	17
6	Declaração de autoria	18

1 Introdução

A disciplina ISW055 — **Introdução à Computação em Nuvem**, do curso de Tecnologia em Sistemas Inteligentes da Fatec Pompeia, apresenta os fundamentos de desenvolvimento e operação de sistemas distribuídos: microsserviços, contêineres, armazenamento de objetos, autenticação, controle de acesso e observabilidade. O fio condutor do bimestre foi um único projeto que cresceu semana a semana: um catálogo de filmes do ator Tom Hanks, iniciado como uma aplicação simples e evoluído até um sistema composto por vários serviços rodando em contêineres.

Durante o período foram realizadas seis atividades, começando por um nivelamento em sala (agenda telefônica em Flask) e o próprio catálogo com consumo da API do TMDb e persistência em MariaDB. Em seguida o login foi desacoplado em um microsserviço de autenticação, o controle de acesso foi endurecido com um modelo de permissões por papel (RBAC), foram adicionados logs e auditoria com Redis Streams e, por fim, o perfil do usuário ganhou upload de foto armazenada no MinIO. Todo o código está versionado em um único repositório público no GitHub, publicado também em um ambiente de produção via Docker Compose.

Este relatório presta contas desse período: apresenta a metodologia usada, um quadro com a data planejada e a data realizada de cada atividade, uma ficha detalhada por entrega com evidências verificáveis (commit, data e hora, e print do resultado) e, ao final, as considerações sobre o que foi aprendido e o que seria feito diferente. O objetivo é documentar, com rastreabilidade e honestidade, o que foi construído e comprovado ao longo do bimestre.

2 Metodologia

Todas as atividades foram desenvolvidas no mesmo repositório público do GitHub ([mateussantospereira/filmes-tom-hanks](https://github.com/mateussantospereira/filmes-tom-hanks)), como continuação de um único projeto. O ambiente de desenvolvimento usou Bun como runtime, o framework Hono para as APIs, MariaDB como banco relacional, Docker Compose para orquestração local e Portainer para o deploy em produção; o armazenamento de arquivos ficou no MinIO e os logs de auditoria no Redis, consumidos por um serviço dedicado de logs. Cada atividade foi implementada, testada localmente e, quando aplicável, publicada no ambiente de produção usado para as demonstrações.

A data e a hora de cada entrega foram extraídas do histórico de commits do repositório com `git log --format="%h %ad"` (formato DD/MM/AAAA HH:MM) e conferidas na página do commit no GitHub. Para cada atividade, o relatório traz duas evidências complementares: um print de terminal com o comando `git log` (hash, data/hora e repositório) e a captura da página do commit no GitHub; além do print do resultado do sistema em execução. Os serviços são públicos em <https://mateus-pereira-isw055.lapps.studio>, e o README do repositório documenta cada atividade e menciona o professor (github.com/siriani).

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência leve no navegador (localStorage).	07/08/2026	07/08/2026 (em sala)	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	18/08/2026 17:48	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	25/09/2026 16:21	entregue com atraso
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	06/10/2026 11:18	entregue com atraso
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	06/10/2026 18:30	entregue com atraso
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	06/10/2026 19:50	entregue com atraso

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência leve no navegador (localStorage).
Data planejada	07/08/2026
Data realizada	07/08/2026 (em sala)
Situação	entregue
Evidência	Realizada em sala — print do sistema rodando localmente
Link	—

O que foi feito. A primeira atividade foi um nivelamento em sala de aula: uma agenda telefônica simples em Flask com templates Jinja, permitindo cadastrar, listar, editar e remover contatos. Como era um exercício de ambientação, a persistência foi mantida propositalmente leve — sem banco de dados, os contatos eram guardados no `localStorage` do navegador. O objetivo era familiarizar a turma com o padrão rota → função → template e com o loop de edição/execução em Python antes de começar o projeto principal.



```
print pendente — insira arquivo: "prints/..." na ficha da atividade 1
```

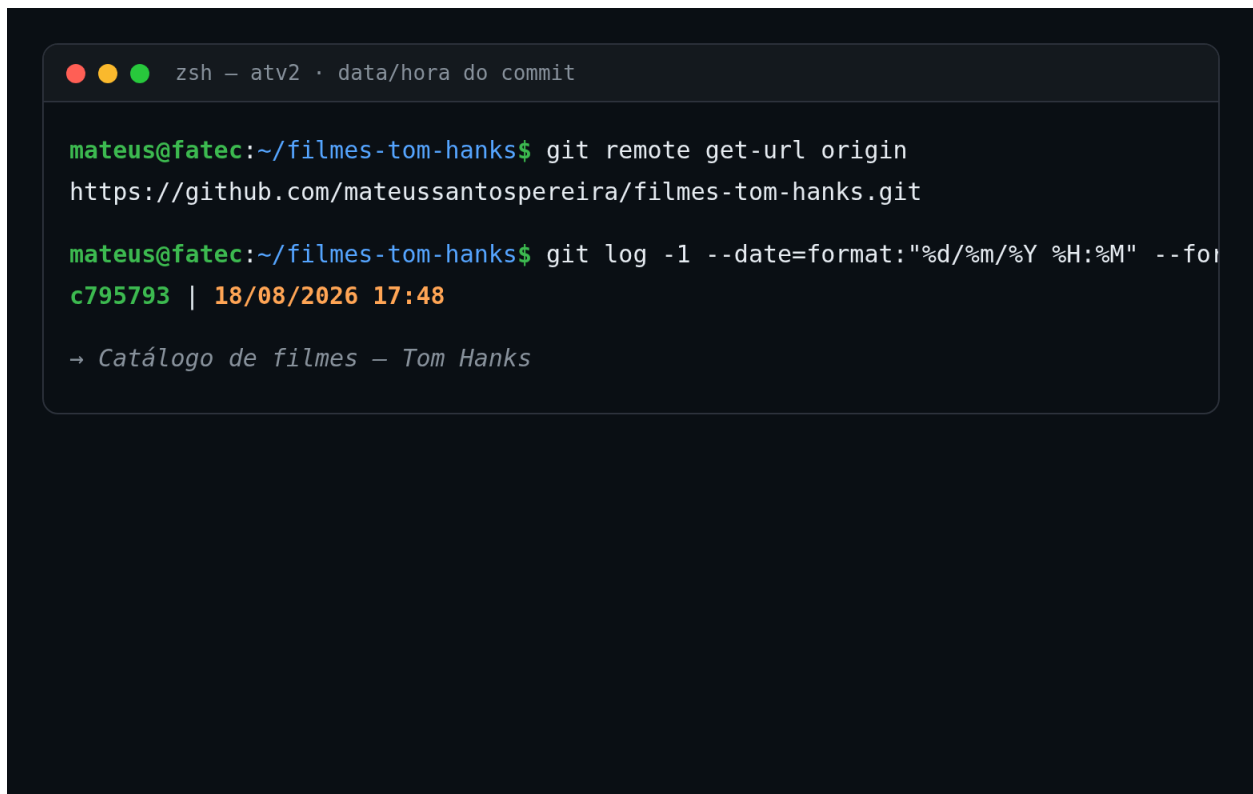
Figura 1: Atividade 1 — evidência da entrega (print do sistema rodando — pendente)

Dificuldades e como foram resolvidas. Por ser um nivelamento, não houve dificuldade técnica relevante; o código não foi versionado no GitHub na ocasião, apenas executado em sala. O print de comprovação do sistema rodando ficou pendente e será anexado a esta ficha assim que recuperado.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	18/08/2026 17:48
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/c795793

O que foi feito. Foi criado o repositório do projeto e o catálogo de filmes do Tom Hanks: a aplicação consome a API do TMDb para buscar e listar os filmes do ator, persiste os dados no MariaDB e oferece uma área logada em que cada usuário tem seus próprios favoritos (segregação por usuário). O docker-compose foi montado para rodar aplicação e banco juntos, com deploy via Portainer. O README documenta a arquitetura, o uso e o endereço de produção, e menciona o professor.



```
zsh - atv2 · data/hora do commit

mateus@fatec:~/filmes-tom-hanks$ git remote get-url origin
https://github.com/mateussantospereira/filmes-tom-hanks.git

mateus@fatec:~/filmes-tom-hanks$ git log -1 --date=format:"%d/%m/%Y %H:%M" --for
c795793 | 18/08/2026 17:48

→ Catálogo de filmes — Tom Hanks
```

Figura 2: Atividade 2 — evidência: commit c795793 (terminal, 18/08/2026 17:48)

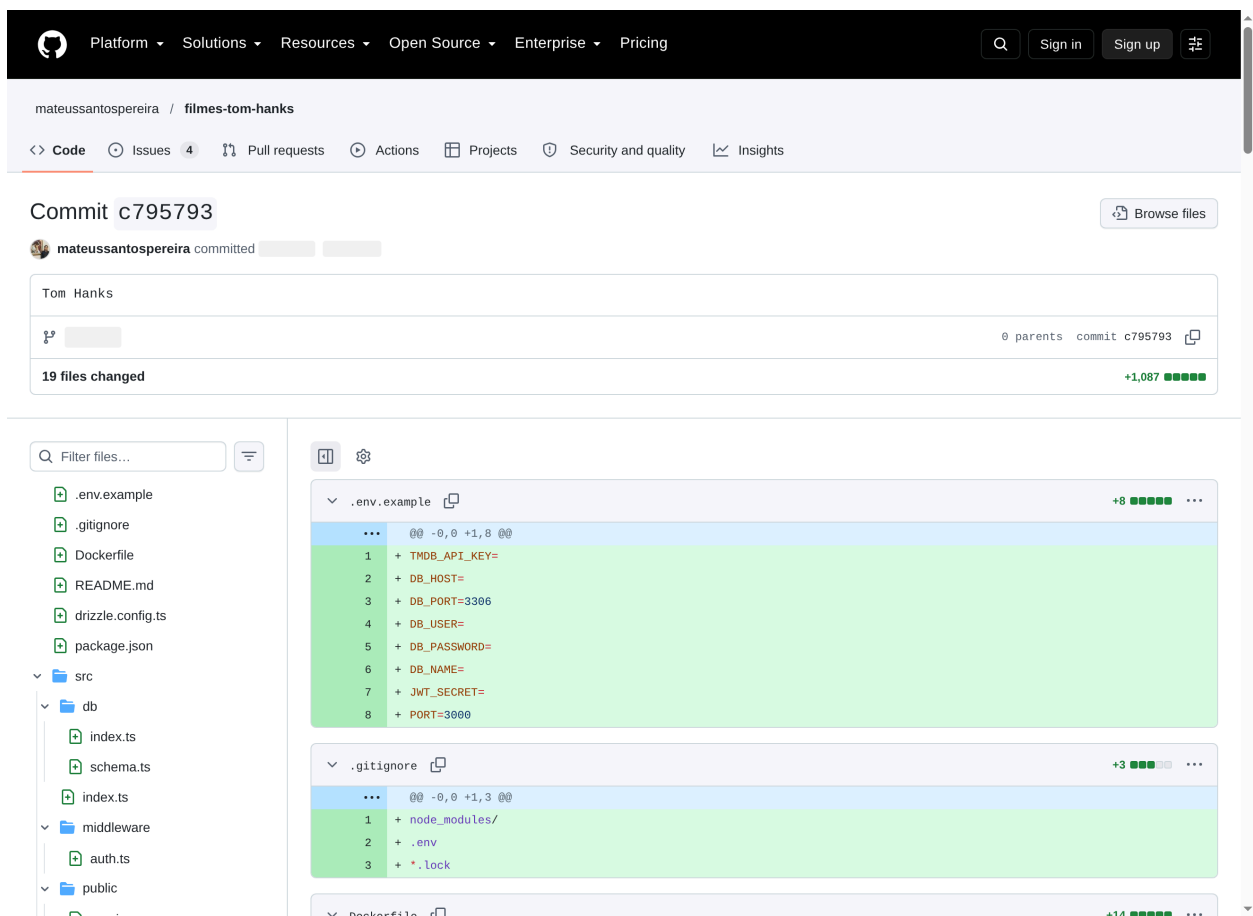


Figura 3: Atividade 2 — evidência: página do commit no GitHub

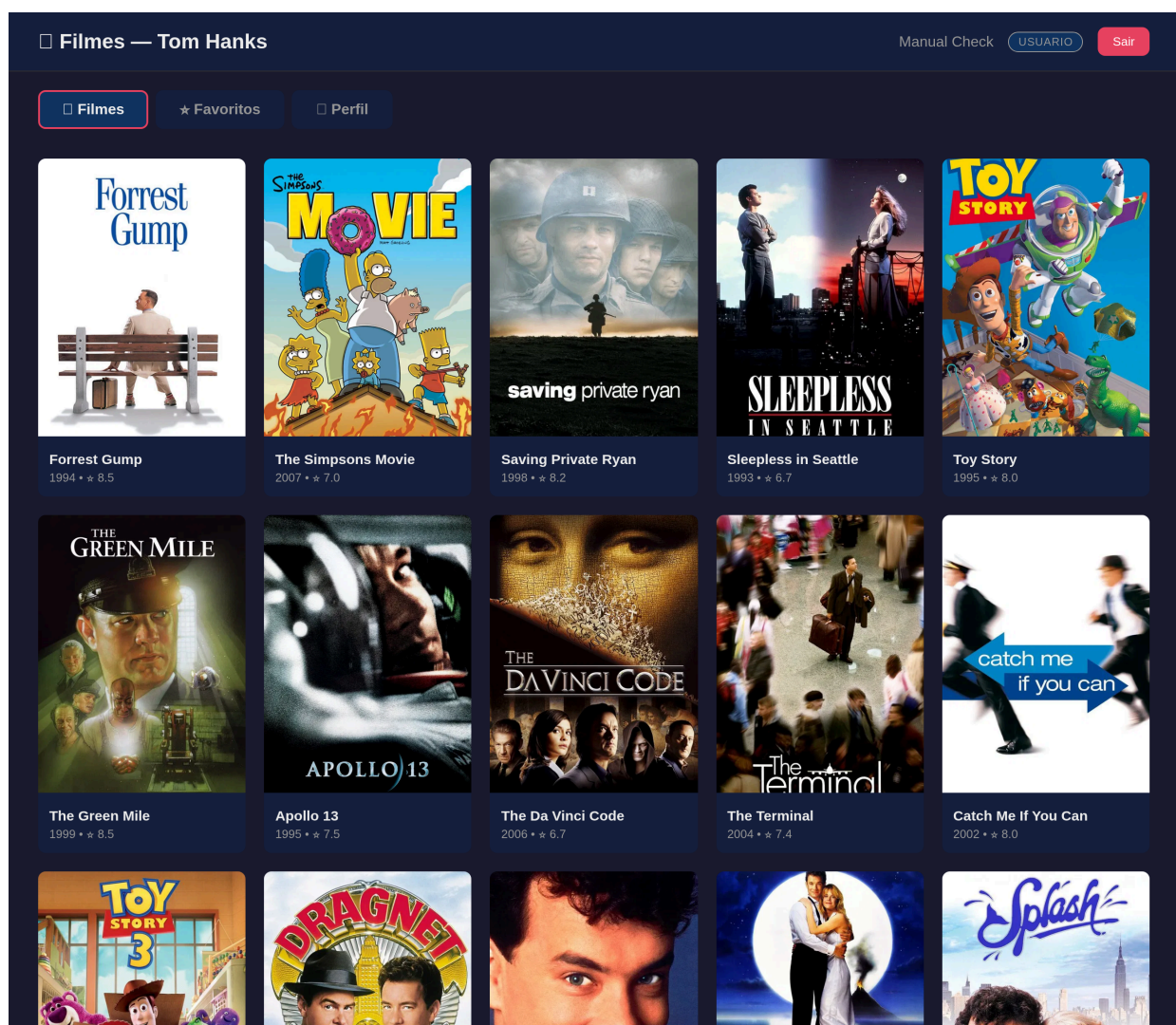


Figura 4: Atividade 2 — resultado: catálogo logado em produção

Dificuldades e como foram resolvidas. A primeira montagem do ambiente (binários, dependências e rede do Docker) consumiu ajustes no Dockerfile; a solução foi isolar o lockfile e o compose em commits dedicados para o build ficar determinístico e reproduzível no Portainer.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	25/09/2026 16:21
Situação	entregue com atraso
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/d08ced9

O que foi feito. A autenticação foi extraída do catálogo para um serviço à parte (auth-service), que passou a ser o único dono da tabela de usuários e da lógica de senha, JWT e papel. O catálogo virou um gateway: as rotas de login, cadastro, esqueci minha senha e reset repassam a chamada ao auth-service

pela rede interna do Docker (AUTH_SERVICE_URL). O fluxo de recuperação de senha envia um e-mail real via Mailtrap, e o docker-compose.yml documenta os dois serviços e suas dependências.

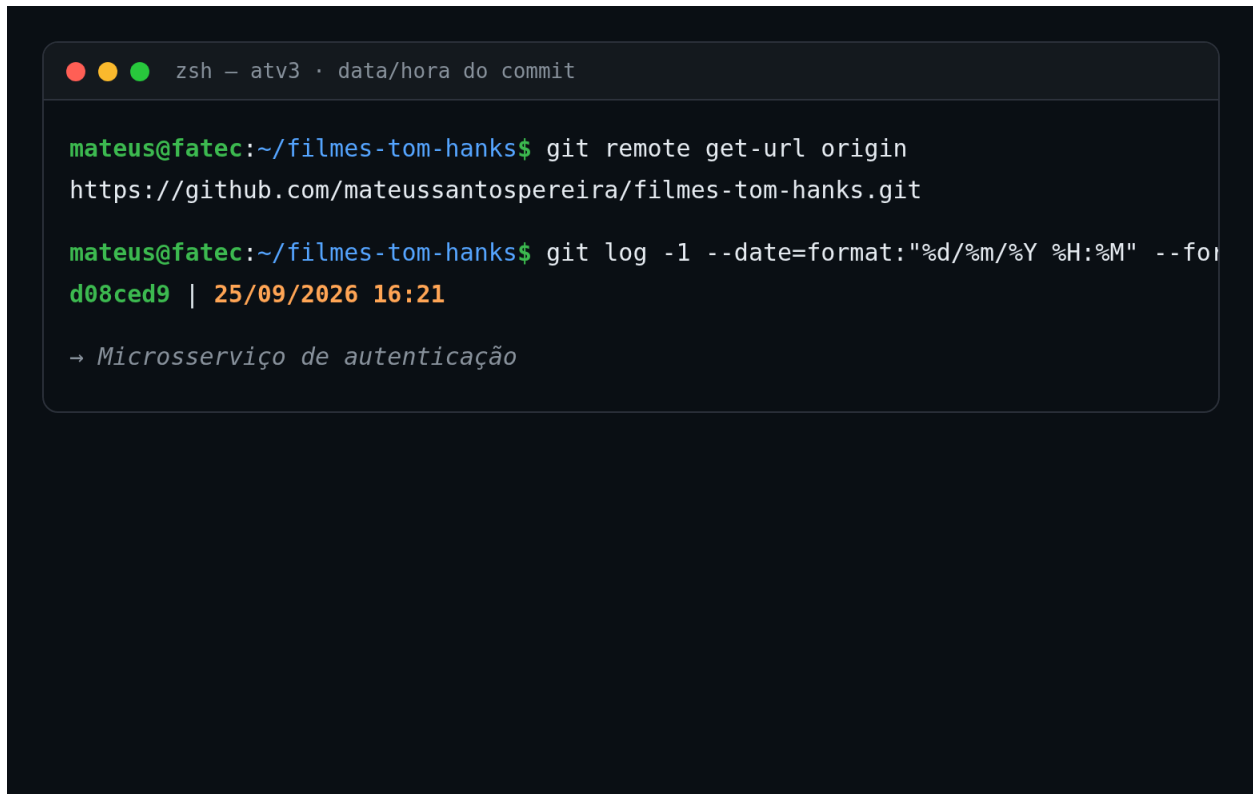
A terminal window with a dark background and light-colored text. The window title bar shows three colored circles (red, yellow, green) followed by the text 'zsh - atv3 · data/hora do commit'. The terminal content shows a user 'mateus@fatec' in the directory '~/filmes-tom-hanks' running two git commands. The first command is 'git remote get-url origin', which returns 'https://github.com/mateussantospereira/filmes-tom-hanks.git'. The second command is 'git log -1 --date=format:"%d/%m/%Y %H:%M" --for', which returns 'd08ced9 | 25/09/2026 16:21'. Below the output, there is a line of text '→ Microserviço de autenticação'.

Figura 5: Atividade 3 — evidência: commit d08ced9 (terminal, 25/09/2026 16:21)

The screenshot shows a GitHub commit page for the repository 'mateussantospereira / filmes-tom-hanks'. The commit hash is 'd08ced9'. The commit message is in Portuguese and describes a refactor of the authentication service into a microservice. It mentions that the previous version was a monolith where the catalog, login, and authentication were all in one place. The new version separates these into different containers that communicate via Docker. It also lists several new features: roles for 'usuario' and 'admin', password recovery via email, a 'forgot password' feature, and a 'reset tokens' endpoint. The commit was made by 'mateussantospereira'.

Platform Solutions Resources Open Source Enterprise Pricing

mateussantospereira / filmes-tom-hanks

Code Issues 4 Pull requests Actions Projects Security and quality Insights

Commit d08ced9

mateussantospereira committed

Atividade 3: extrai a autenticao para um microservico a parte

A atividade 2 era um monolito: o catalogo tinha login, cadastro, emissao e validacao de JWT, consulta de senha e a tabela de usuarios. Mexer na autenticao exigia derrubar a aplicacao inteira, e uma falha no login derrubava tambem o catalogo de filmes.

Agora sao dois containers que conversam pela rede do Docker:

```
navegador -> app (8222, publico) -> http://auth-service:3000 -> MariaDB
```

O catalogo virou o unico ponto de entrada publico. O auth-service nao tem 'ports:' e so existe dentro da rede 'interna', alcancavel pelo nome do servico. O JWT_SECRET saiu do catalogo e ficou so no servico que emite: segredo do lado de quem valida nao protege nada, porque daria para forjar um token de admin.

O preco da separacao e aceito de proposito: se o auth-service cair, tudo que depende de sessao responde 503 e o catalogo continua no ar. Com o monolito, uma falha no login derrubava tudo junto.

Novidades da atividade:

- papeis 'usuario' e 'admin'; o papel vem do banco, nao do token, entao promover alguem tem efeito no request seguinte
- recuperacao de senha por e-mail (nodemailer), com link de 30 minutos entregue de verdade por SMTP: Mailtrap no dev, Brevo na producao
- tabela 'reset_tokens' (token, usuario_id, criado_em, expira_em, usado), com criado_em/expira_em calculados pelo banco para nao depender do relógio do container
- validacao do link em tres etapas (existe, nao expirou, nao usado), com consumo atomico do token para o mesmo link nao valer duas vezes
- "esqueci minha senha" responde a mesma coisa para e-mail cadastrado e inexistente, para nao dar para descobrir quem tem conta
- exclusao de comentario restrita ao dono ou a admin (403 para os demais)

Infra:

Figura 6: Atividade 3 — evidência: página do commit no GitHub

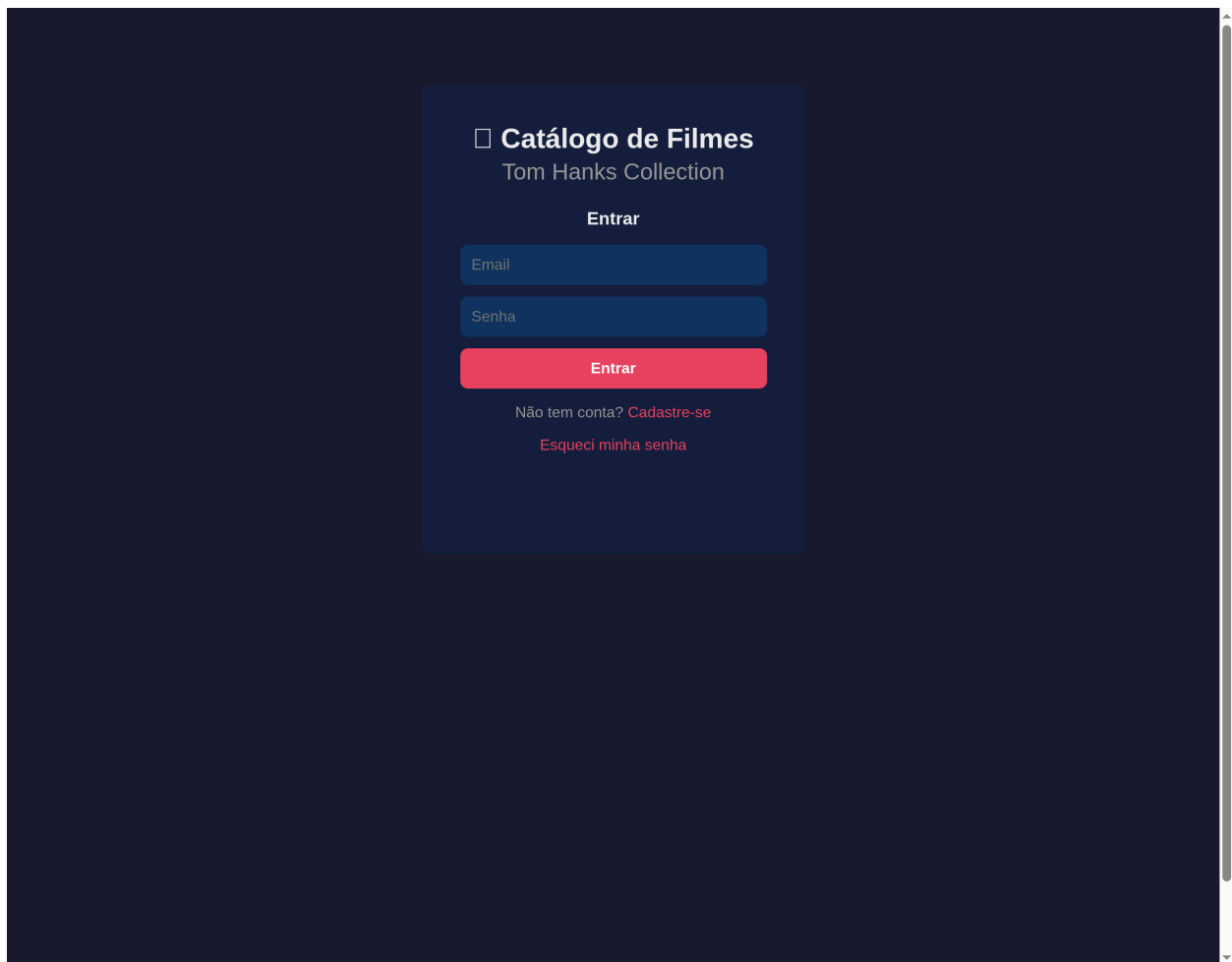


Figura 7: Atividade 3 — resultado: tela de login em produção

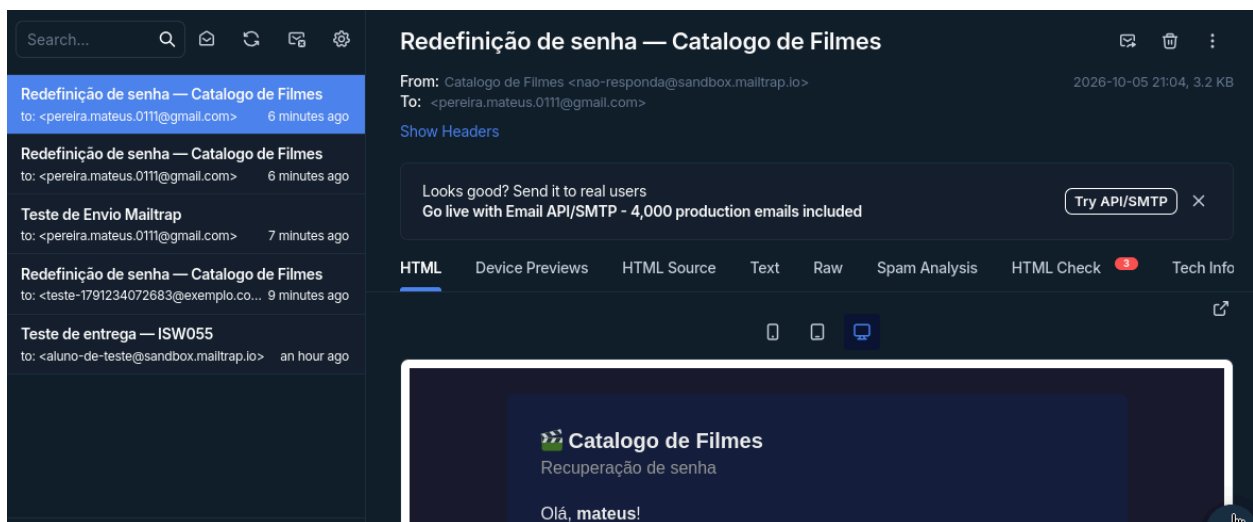


Figura 8: Atividade 3 — resultado: e-mail de recuperação de senha (Mailtrap)

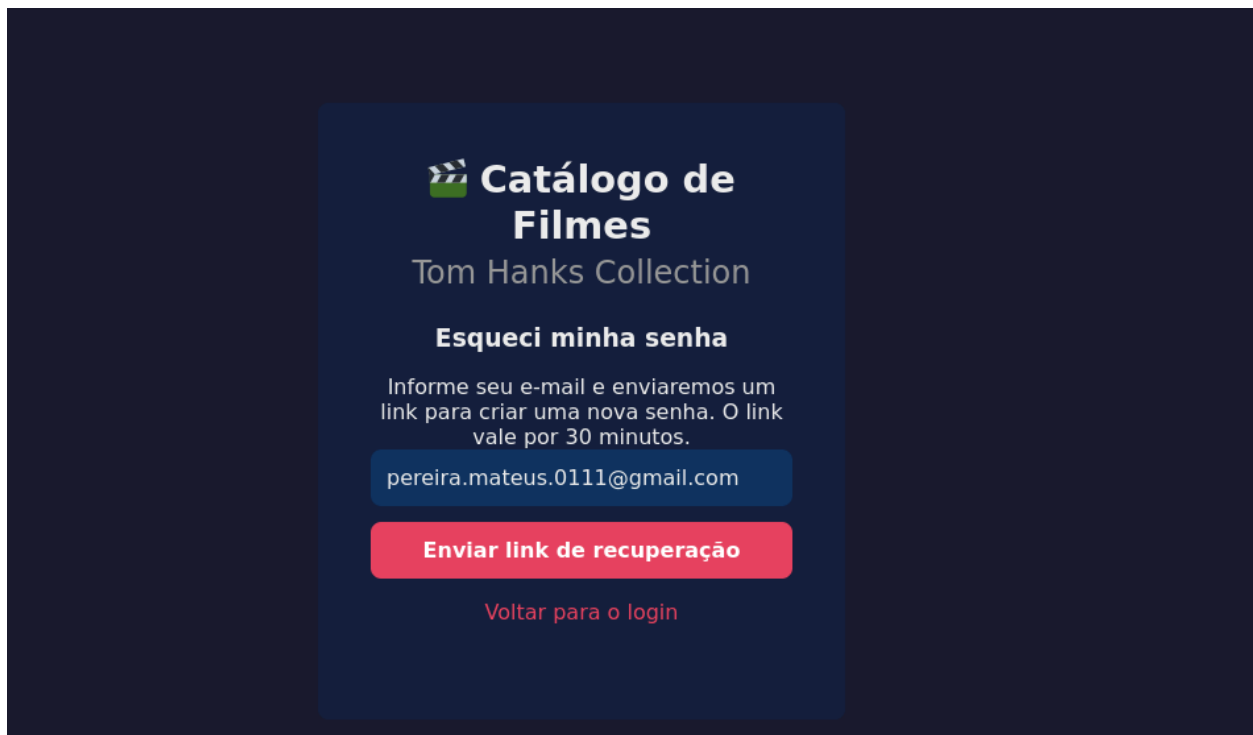


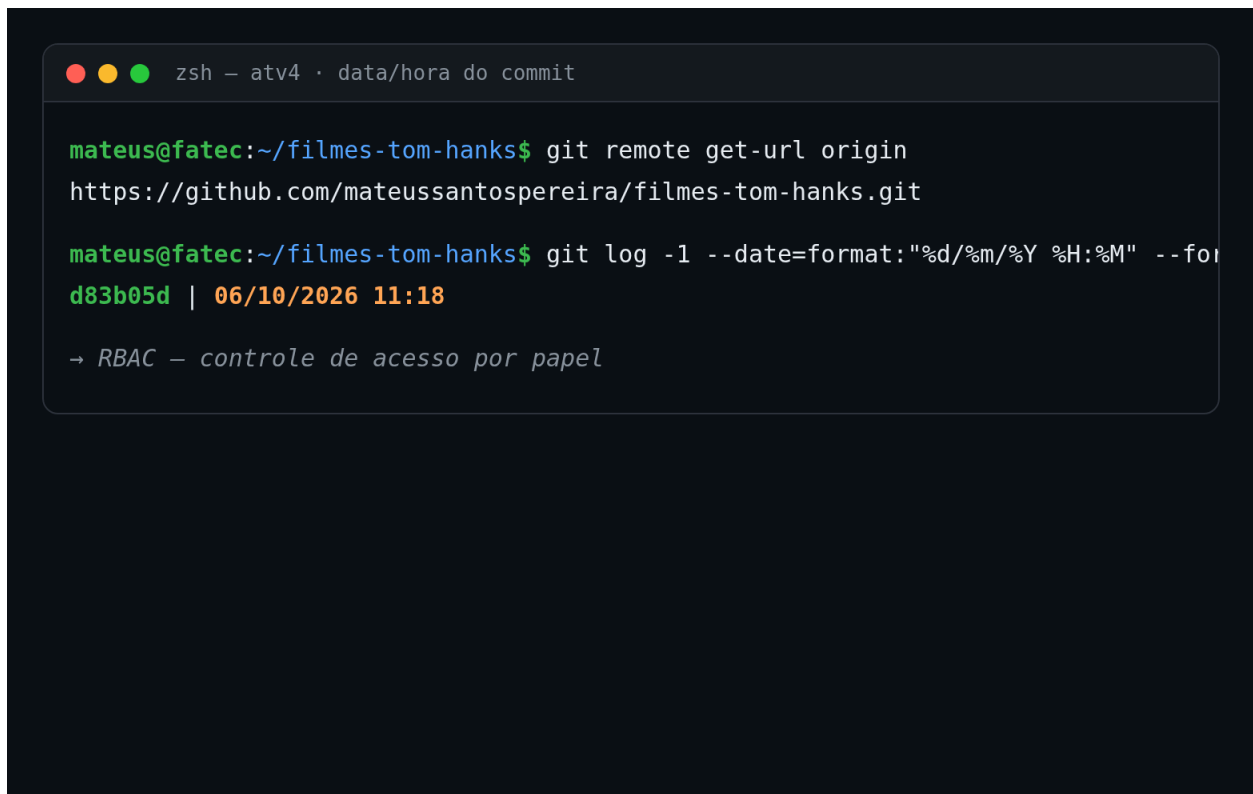
Figura 9: Atividade 3 — resultado: fluxo de redefinição de senha

Dificuldades e como foram resolvidas. Sincronizar dois serviços em uma rede interna sem expor o auth-service para fora exigiu cuidar da variável `AUTH_SERVICE_URL` (nome do serviço, não localhost) e de timeouts para não segurar requisições do usuário com o serviço travado. O ajuste fino dos defaults de produção e das dependências (mysql2) foi concluído na semana seguinte à implementação principal, o que atrasou a entrega em relação à data planejada.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	06/10/2026 11:18
Situação	entregue com atraso
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/d83b05d

O que foi feito. O campo role deixou de ser decorativo e passou a decidir permissões no servidor: foi criado um mapa explícito de permissões (`auth-service/src/auth/permissions.ts`), com admin como superconjunto de usuario e decisão sempre **fail-closed**. `GET /me` devolve as permissões resolvidas, e o catálogo verifica `permissions.includes(...)` em vez de comparar `role === "admin"`. Ações exclusivas de admin (apagar comentário de terceiro, listar usuários, alterar papel) negam com 403 para usuário comum; um bug de XSS armazenado em comentários também foi corrigido trocando `innerHTML` por `textContent`.



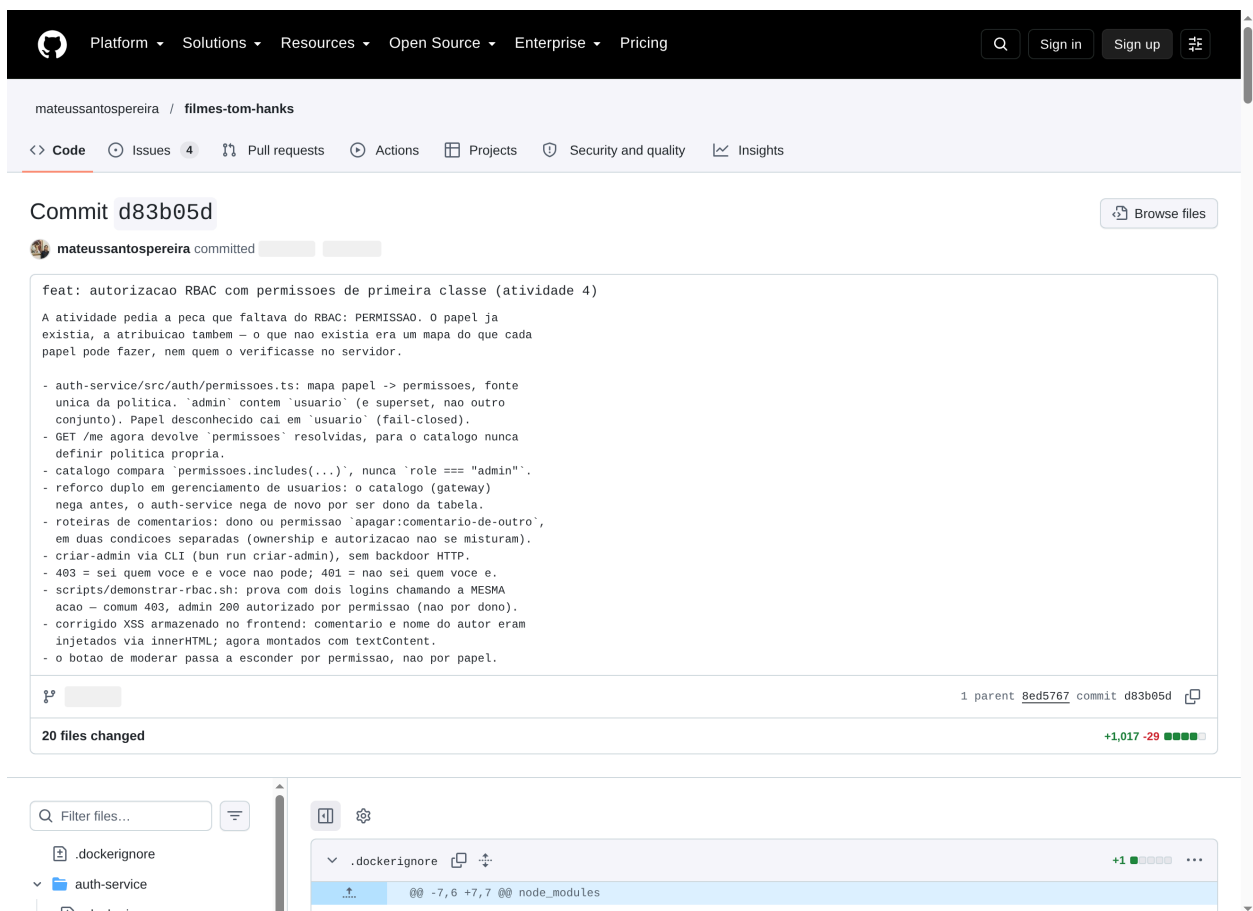
```
zsh - atv4 · data/hora do commit

mateus@fatec:~/filmes-tom-hanks$ git remote get-url origin
https://github.com/mateussantospereira/filmes-tom-hanks.git

mateus@fatec:~/filmes-tom-hanks$ git log -1 --date=format:"%d/%m/%Y %H:%M" --for
d83b05d | 06/10/2026 11:18

→ RBAC – controle de acesso por papel
```

Figura 10: Atividade 4 — evidência: commit d83b05d (terminal, 06/10/2026 11:18)



Platform Solutions Resources Open Source Enterprise Pricing

mateussantospereira / filmes-tom-hanks

<> Code Issues 4 Pull requests Actions Projects Security and quality Insights

Commit d83b05d

mateussantospereira committed

feat: autorizacao RBAC com permissoes de primeira classe (atividade 4)

A atividade pedia a peça que faltava do RBAC: PERMISSAO. O papel já existia, a atribuição também — o que não existia era um mapa do que cada papel pode fazer, nem quem o verificasse no servidor.

- auth-service/src/auth/permissoes.ts: mapa papel -> permissoes, fonte única da política. 'admin' contém 'usuario' (e superset, não outro conjunto). Papel desconhecido cai em 'usuario' (fail-closed).
- GET /me agora devolve 'permissoes' resolvidas, para o catálogo nunca definir política própria.
- catálogo compara 'permissoes.includes(...)', nunca 'role === "admin"'. reforço duplo em gerenciamento de usuários: o catálogo (gateway) nega antes, o auth-service nega de novo por ser dono da tabela.
- rotas de comentários: dono ou permissão 'apagar:comentario-de-outro', em duas condições separadas (ownership e autorizacao não se misturam).
- criar-admin via CLI (bun run criar-admin), sem backdoor HTTP.
- 403 = sei quem você é e você não pode; 401 = não sei quem você é.
- scripts/demonstrar-rbac.sh: prova com dois logins chamando a MESMA ação — comum 403, admin 200 autorizado por permissão (não por dono).
- corrigido XSS armazenado no frontend: comentário e nome do autor eram injetados via innerHTML; agora montados com textContent.
- o botão de moderar passa a esconder por permissão, não por papel.

1 parent 8ed5767 commit d83b05d

20 files changed +1,017 -29

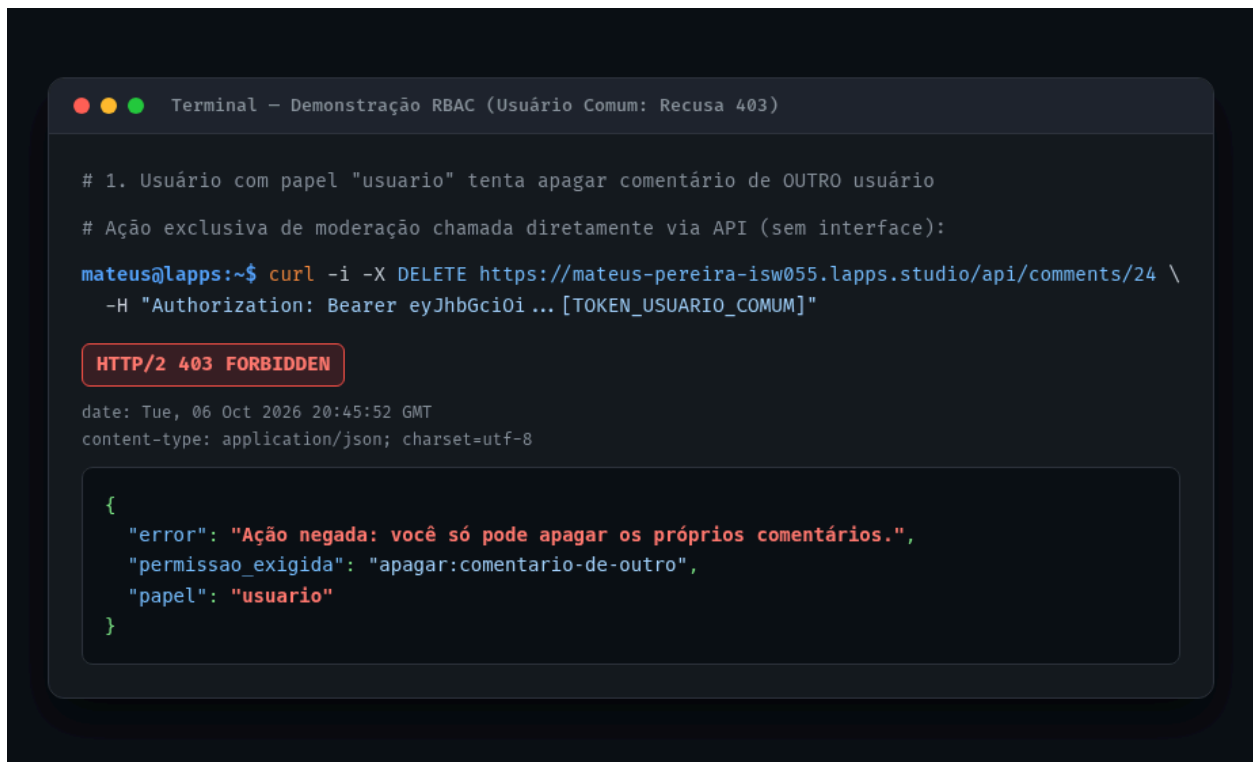
Filter files...

- .dockerignore
- auth-service
- .dockerignore

.dockerignore

@@ -7,6 +7,7 @@ node_modules

Figura 11: Atividade 4 — evidência: página do commit no GitHub



Terminal — Demonstração RBAC (Usuário Comum: Recusa 403)

```
# 1. Usuário com papel "usuario" tenta apagar comentário de OUTRO usuário
# Ação exclusiva de moderação chamada diretamente via API (sem interface):
mateus@lapps:~$ curl -i -X DELETE https://mateus-pereira-isw055.lapps.studio/api/comments/24 \
-H "Authorization: Bearer eyJhbGciOi ... [TOKEN_USUARIO_COMUM]"
```

HTTP/2 403 FORBIDDEN

date: Tue, 06 Oct 2026 20:45:52 GMT
content-type: application/json; charset=utf-8

```
{
  "error": "Ação negada: você só pode apagar os próprios comentários.",
  "permissao_exigida": "apagar:comentario-de-outro",
  "papel": "usuario"
}
```

Figura 12: Atividade 4 — resultado: 403 para usuário comum



Terminal — Demonstração RBAC (Administrador: Sucesso 200 por Permissão)

```
# 2. Administrador com papel "admin" tenta apagar o MESMO comentário de outro usuário
# O servidor autoriza a moderação pela permissão "apagar:comentario-de-outro":
mateus@lapps:~$ curl -i -X DELETE https://mateus-pereira-isw055.lapps.studio/api/comments/24 \
-H "Authorization: Bearer eyJhbGciOi ... [TOKEN_ADMIN]"
```

HTTP/2 200 OK

date: Tue, 06 Oct 2026 20:45:54 GMT
content-type: application/json; charset=utf-8

```
{
  "message": "Comentário removido",
  "autorizacao": "permissao:apagar:comentario-de-outro"
}
```

Figura 13: Atividade 4 — resultado: a mesma ação autorizada para admin

Dificuldades e como foram resolvidas. A entrega acumulou atraso por ter sido desenvolvida junto com a montagem das outras atividades. A principal dificuldade técnica foi provar a regra dos dois lados: a demonstração usa uma terceira conta como alvo, para que a exclusão pelo admin aconteça por **permissão** e não por **ownership**. O XSS armazenado foi descoberto durante a revisão do frontend e corrigido antes da entrega.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	06/10/2026 18:30
Situação	entregue com atraso
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/dca3a82

O que foi feito. Foi criado um serviço dedicado de logs (log-service) que consome eventos de um Redis Streams e os disponibiliza para consulta: login bem-sucedido, ações sensíveis (como troca de papel) e tentativas negadas com 403. Um middleware global de auditoria captura qualquer `acao_negada` na borda do catálogo e a publica no stream; o admin consulta os eventos pela interface de auditoria do sistema. Nas atividades seguintes o mesmo mecanismo passou a registrar também `upload_foto` e `editar_perfil`.

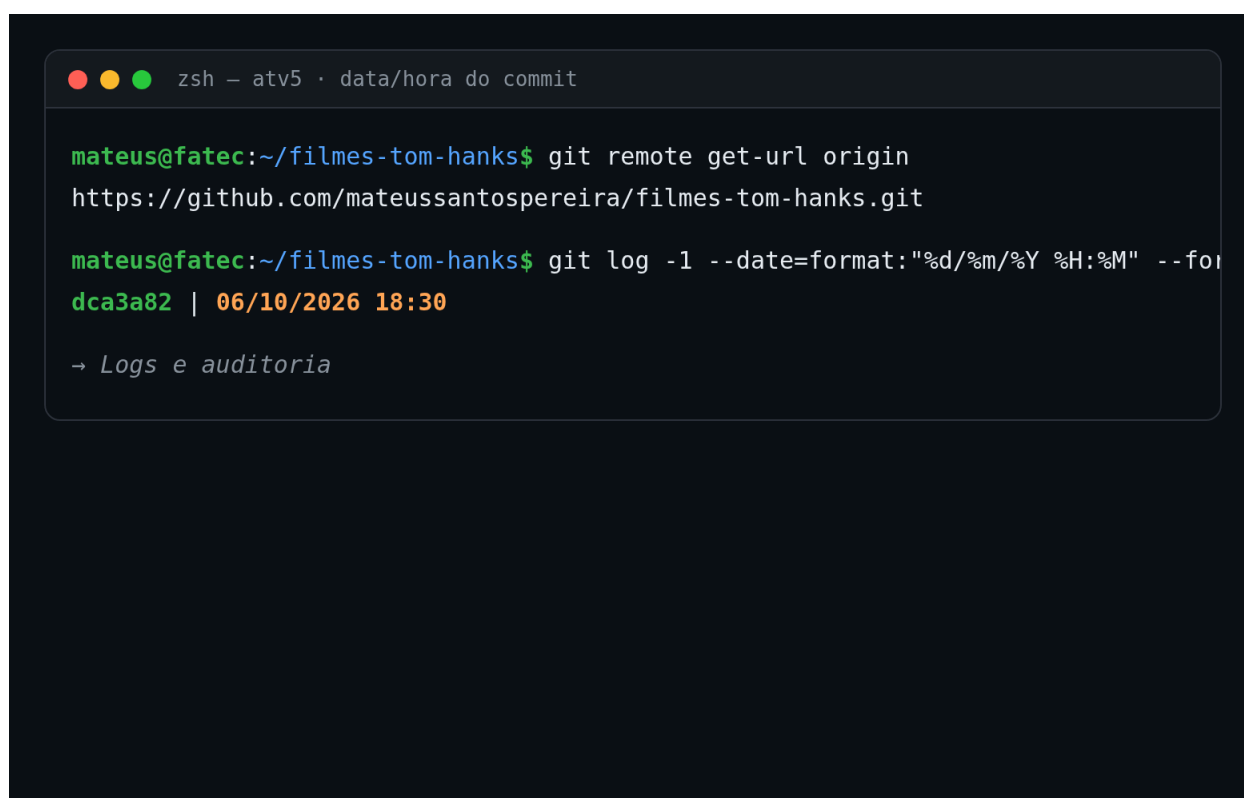
A terminal window with a dark background. The title bar shows three colored circles (red, yellow, green) and the text 'zsh - atv5 · data/hora do commit'. The terminal content shows a user 'mateus@fatec' in the directory '~/filmes-tom-hanks' running 'git remote get-url origin', which returns 'https://github.com/mateussantospereira/filmes-tom-hanks.git'. Then, the user runs 'git log -1 --date=format:"%d/%m/%Y %H:%M" --for dca3a82', which returns '06/10/2026 18:30'. Below the output, there is a line '→ Logs e auditoria'.

Figura 14: Atividade 5 — evidência: commit dca3a82 (terminal, 06/10/2026 18:30)

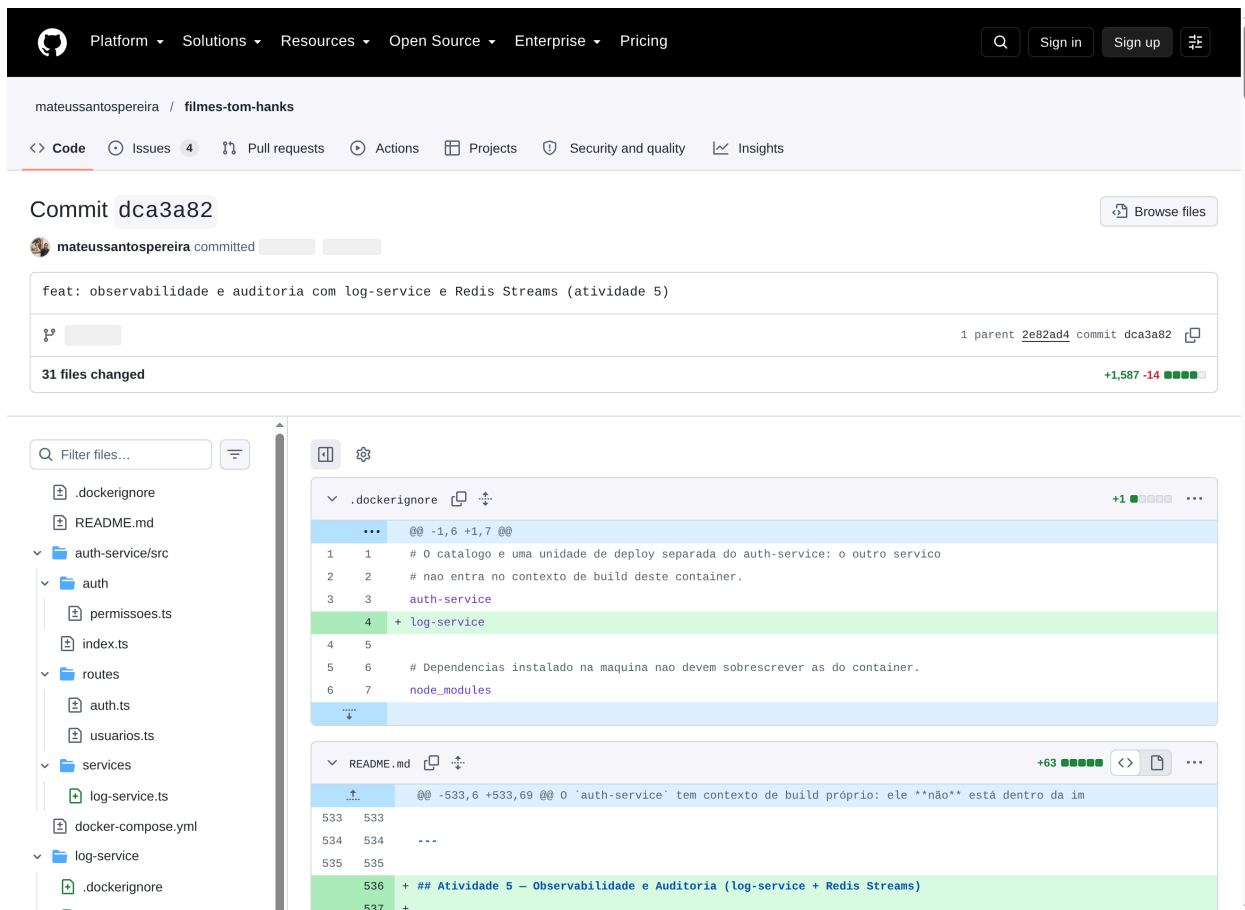


Figura 15: Atividade 5 — evidência: página do commit no GitHub

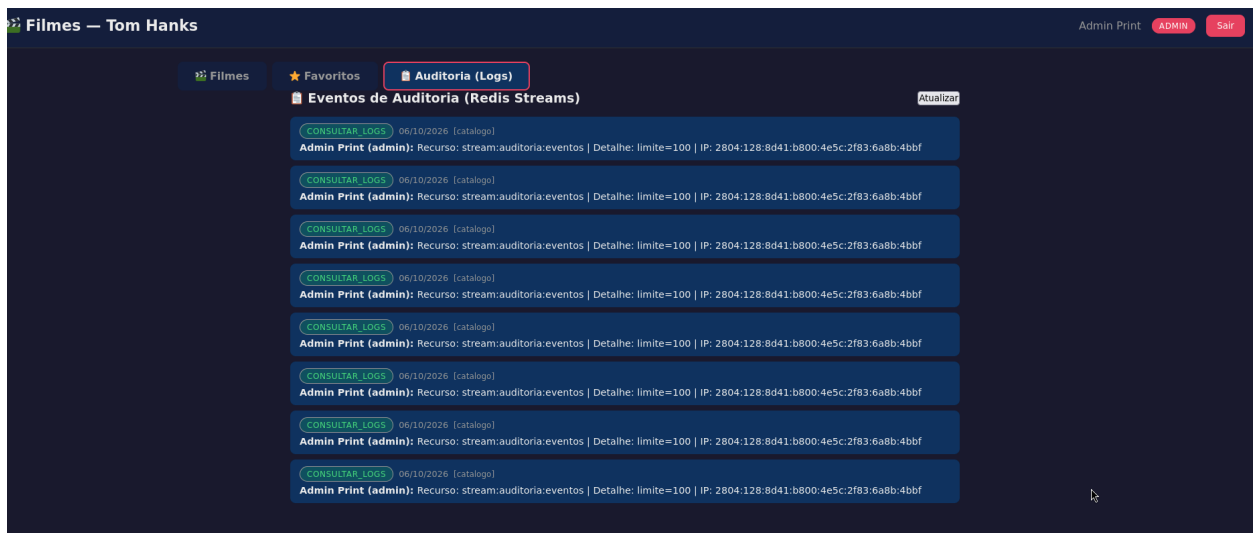


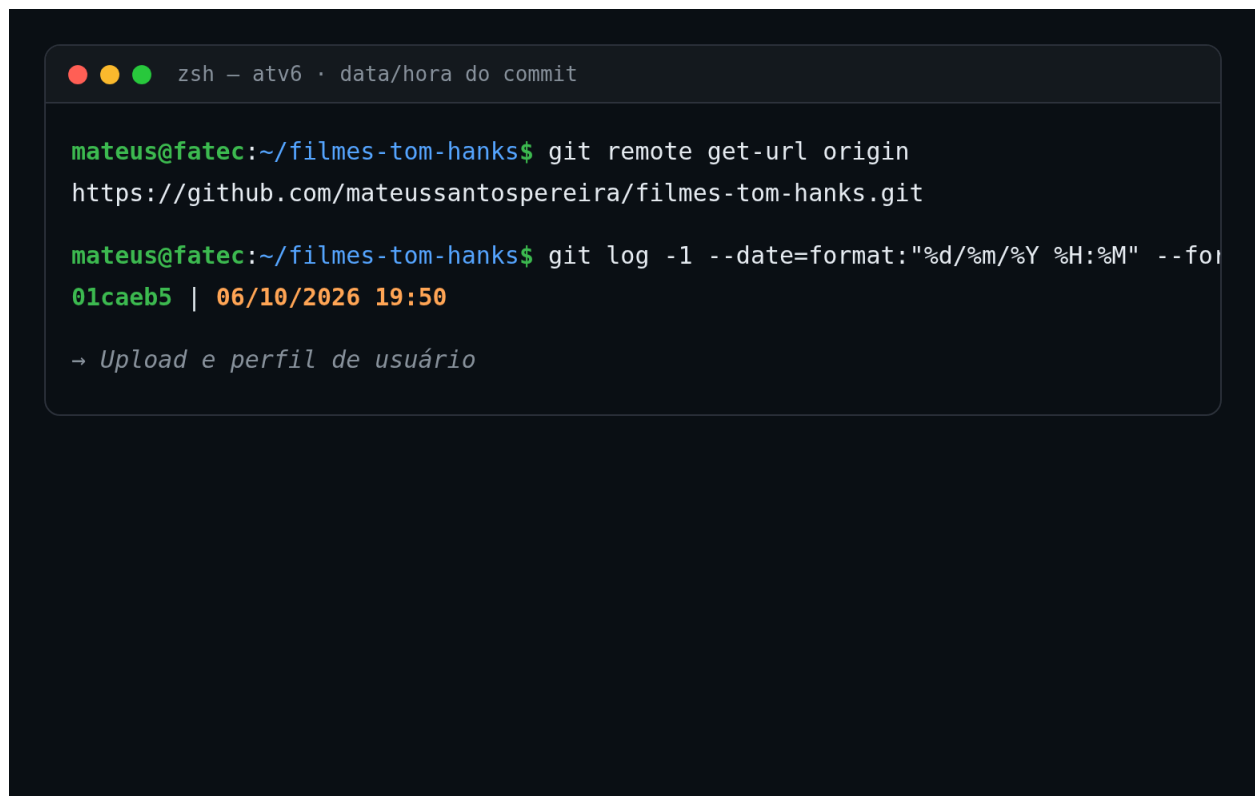
Figura 16: Atividade 5 — resultado: consulta dos logs de auditoria pelo admin

Dificuldades e como foram resolvidas. O Redis e o log-service não persistem volume, então o stream de auditoria zera a cada redeploy — comportamento esperado e documentado, mas que exigiu a demonstração ser rodada contra o ambiente com os serviços ativos. Também foi necessário versionar os estáticos (?v=) para o cache do CDN não servir script antigo.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	06/10/2026 19:50
Situação	entregue com atraso
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/mateussantospereira/filmes-tom-hanks/commit/01caeb5

O que foi feito. A página de perfil do usuário ganhou bio e foto: o arquivo enviado vai para o bucket privado perfis do MinIO (com volume persistente no compose), a referência fica no MariaDB e a foto volta ao navegador por uma rota autenticada do catálogo (GET /api/perfil/:id/foto). O upload é validado no servidor (apenas imagem — 415; máximo 2 MB — 413). A identidade vem do token, nunca do ID no corpo/URL: editar ou fotografar o perfil de outra pessoa devolve 403 (permissao_exigida: "editar:perfil-proprio"), e esse evento entra na auditoria como acao_negada.



```
zsh — atv6 · data/hora do commit

mateus@fatec:~/filmes-tom-hanks$ git remote get-url origin
https://github.com/mateussantospereira/filmes-tom-hanks.git

mateus@fatec:~/filmes-tom-hanks$ git log -1 --date=format:"%d/%m/%Y %H:%M" --for
01caeb5 | 06/10/2026 19:50

→ Upload e perfil de usuário
```

Figura 17: Atividade 6 — evidência: commit 01caeb5 (terminal, 06/10/2026 19:50)

mateussantospereira / filmes-tom-hanks

<> Code Issues 4 Pull requests Actions Projects Security and quality Insights

Commit 01caeb5

mateussantospereira committed

Atividade 6: foto de perfil no MinIO

- serviço minio.ts: cliente do object storage (bucket 'perfis' privado, chave versionada perfis/<id>/avatar-<ts>.<ext>, magic bytes PNG/JPEG/GIF/WebP)
- rotas /api/perfil: GET proprio, GET por id (nome via novo endpoint perfil-publico do auth-service), PATCH bio e POST foto (so o dono, 403 se nao; nada confia em id do corpo/URL), GET /:id/foto servindo os bytes
- tabela perfis (bio + foto_chave = so a referencia) + sql/atividade-6.sql + db:migrate idempotente (nunca db:push em producao)
- frontend: aba 'Perfil' (avatar, bio editavel, upload, favoritos da atividade 2, autor de comentario abre o perfil) + app.js?v=3
- docker-compose: serviço minio sem ports: (rede interna), volume minio-data; minio/minio removido dos registries (arquivado) -> imagem comunitaria elastio/minio (ultima release open-source), documentado no README
- auditoria (atv 5): eventos upload_foto e editar_perfil; acao_negada 403 ja era capturado
- scripts/demonstrar-perfil.sh + fixture avatar-teste.png (100% verde local)

1 parent a0fb318 commit 01caeb5

17 files changed +1,413 -6

Filter files...

- README.md
- auth-service/src/routes
- usuarios.ts
- docker-compose.yml
- drizzle.config.ts
- package.json

diff --git a/README.md b/README.md

```

697 697
698 698 ---
699 699
700 + ## Atividade 6 – Armazenamento de objetos (foto de perfil + MinIO)
701 +

```

Figura 18: Atividade 6 — evidência: página do commit no GitHub

Filmes — Tom Hanks

math ADMIN San

Filmes Favoritos Perfil Auditoria (Logs)

math ADMIN

Sua foto não vai para o banco: o arquivo fica no MinIO (object storage) e aqui só a referência.

Sou um estudante de programação

Salvar bio

Trocar foto (PNG/JPEG/GIF/WebP, máx 2 MB)

Filmes favoritos

Forrest Gump The Green Mile Saving Private Ryan Captain Phillips

Figura 19: Atividade 6 — resultado: perfil com foto de upload no MinIO

```
bash - ./scripts/demonstrar-perfil.sh · saída real contra produção

mateus@lapps:~/filmes-tom-hanks$ CATALOGO_URL=https://mateus-pereira-isw055.lapps.studio ./scripts/demonstrar-perfil.sh

# A identidade do usuário NUNCA vem do corpo/URL: o servidor resolve o TOKEN pelo auth-service
# (GET /me) e compara com o id enviado — id de outra pessoa? 403 na borda, sem olhar o conteúdo.

PASSO 6 · TENTATIVA (RECUSADA) DE EDITAR O PERFIL DE OUTREM

✓ comum edita a BIO de outro (PATCH /api/perfil/52) 403

{
  "error": "Ação negada: você só pode editar o seu próprio perfil.",
  "permissao_exigida": "editar:perfil-proprio",
  "papel": "usuario",
  "id_recebido": 52,
  "sua_conta": 51
}

✓ comum envia FOTO para o perfil de outro (POST /api/perfil/52/foto) 403

{
  "error": "Ação negada: você só pode editar o seu próprio perfil.",
  "permissao_exigida": "editar:perfil-proprio",
  "papel": "usuario",
  "id_recebido": 52,
  "sua_conta": 51
}

✓ bio de outra pessoa segue intacta
✓ foto de outra pessoa segue ausente

PASSO 7 · AUDITORIA CAPTURA OS EVENTOS (CONTINUAÇÃO DA ATIVIDADE 5)

# Upload e edição geram eventos novos no mesmo stream do Redis; os 403 viram acao_negada.
# Conferido no log de auditoria (log-service + emissão via /api/logs).

✓ evento 'upload_foto' presente no log de auditoria
✓ evento 'editar_perfil' presente no log de auditoria
✓ evento 'acao_negada' presente no log de auditoria

TODAS AS CONFERENCIAS DA ATIVIDADE 6 PASSARAM (exit 0)
```

Figura 20: Atividade 6 — resultado: recusa de editar perfil alheio (403)

Dificuldades e como foram resolvidas. No primeiro deploy a foto não aparecia: `background-image` no CSS não manda o header `Authorization`, então a rota autenticada devolvia 401. A solução foi carregar a foto com `fetch` enviando o token e montar uma blob URL (`URL.createObjectURL`) para o avatar. Em seguida, uma entrada de cache do CDN armazenada com o código antigo exigiu subir o cache-buster para `?v=5` — o que reforçou a regra de nunca pré-aquecer a URL nova antes do redeploy terminar.

5 Considerações finais

O bimestre mostrou, na prática, a distância entre “fazer funcionar na minha máquina” e “fazer funcionar em produção”. A parte mais difícil foi operar o sistema em contêineres: rede interna entre serviços,

variáveis de ambiente por ambiente, cache de CDN servindo versão velha e serviços sem persistência de volume zerando dados de auditoria. Cada um desses problemas virou uma lição documentada no README e, mais importante, um comportamento que hoje eu já antecipo antes de deplorar.

O ritmo de entrega não foi uniforme: o catálogo saiu antes do prazo, mas o desacoplamento do login e principalmente as atividades de RBAC e auditoria acumularam atraso e foram concluídas na reta final. Se pudesse fazer diferente, dividiria cada entrega em incrementos menores e faria o deploy de produção logo após cada atividade, em vez de concentrar o fechamento das demais no fim do período. Ainda assim, o projeto chegou completo: seis atividades entregues e comprovadas, com demonstrações automatizadas rodando contra o ambiente real.

Para o próximo bimestre, levo a rotina de validação automatizada (scripts de demonstração com saída verificável) e o hábito de registrar dificuldades e soluções à medida que acontecem — foi isso que permitiu que este relatório fosse escrito com datas e evidências reais, e não de memória.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Mateus dos Santos Pereira

Pompeia, 07/10/2026